

A Heterogeneous Graph-based Gate Sizer Integrating Graph Attention Network and Transformer

Jinmo Ahn, Jinoh Cho, Jaemin Seo, Jaeseung Lee, Jakang Lee and Seokhyeong Kang

Department of Electrical Engineering, Pohang University of Science and Technology

Pohang, Republic of Korea

{jinmo.ahn, jinoh.cho, seojm, jae2seung, wkrkd95, shkang}@postech.ac.kr

Abstract—Gate sizing is a critical step in achieving the target power, performance, and area (PPA) in chip design. In recent years, machine learning (ML) methods have recently emerged as a new paradigm for gate sizing. Their promising results have gained attention; however, the practical applicability and performance of existing works is limited by at least one of the following factors: (1) long runtime due to test-time optimization or autoregressive prediction; (2) limited exploration of architectural choices; (3) an overly simplified data representation, known as a homogeneous graph, which merges pins and cells into a single node. To improve both practicality and performance, we introduce a novel ML-based gate sizer, dubbed *DPH-Sizer*, which directly predicts the appropriate gate sizes using a heterogeneous graph that separates cells and their pins into distinct nodes. This heterogeneous graph explicitly captures the relationships between different circuit elements, leading to enhanced performance. Lastly, we propose Inter-Cell and Intra-Cell GAT blocks to explicitly capture both intra-cell and inter-cell information. These are followed by transformer blocks, which are placed at the end of the GAT stack to capture global path-level features. In our experiments, we validate each of the proposed components and demonstrate that *DPH-Sizer* maintains power consumption within 2.0% on average while achieving improvements of 54.3% in timing (WNS) and 1.3% in area metrics.

I. INTRODUCTION

Gate sizing has long been explored as a technique for optimizing PPA of the circuit by assigning the appropriate gate size to each cell from the available library candidates. As this sizing technique is one of the most powerful steps in achieving the target PPA, it has been extensively studied for a long time. Lagrangian relaxation (LR)-based methods [1–8] used to be the *de facto* choice, while dynamic programming (DP) [9–11] and convex programming [12] were also prevalent in the past. In recent years, machine learning (ML) has shed new light on the gate sizing problem.

The mainstream ML-based works [13–19] leverage graph representations with graph neural networks (GNNs) to encode meaningful embeddings through a message-passing scheme. RL-Sizer [13] was the first to employ GNNs to extract states and actions for a Markov Decision Process. However, this reinforcement learning (RL) framework inherently requires a relatively long exploration and exploitation time to achieve satisfactory outcomes. AGD [14] employs GNNs to imitate modern static timing analysis (STA) engines and then uses the GNN-based neural STA engine to obtain gradients for updating

This research was supported by the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2023-00222085, Development of memory module and memory compiler for non-volatile PIM) and by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No.2019R1A5A1027055).

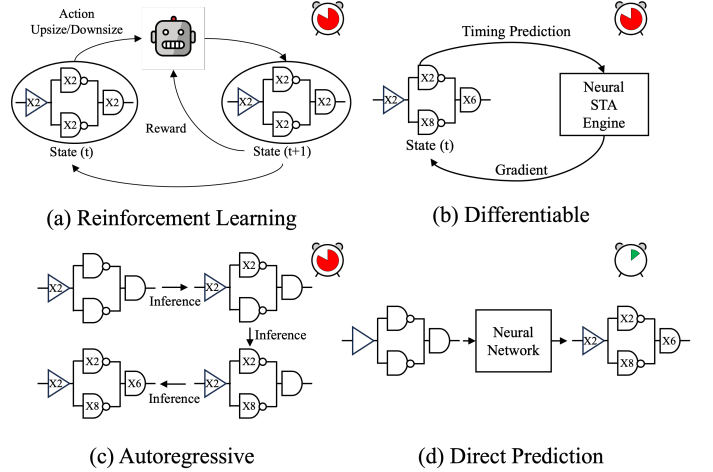


Fig. 1: Illustration of four gate sizing approaches: (a) Reinforcement Learning, (b) Differentiable optimization, (c) Direct Prediction. Each node represents a gate, and its size is indicated next to the node if it has been determined. RL, Differentiable optimization, and Autoregressive methods require multiple inference steps, leading to significantly longer runtimes compared to direct prediction.

gate sizes. Similarly, Ye et al. [15] update gate sizes using gradients, like AGD, but obtain more accurate gradients by proposing a sizing-oriented straight-through estimator (STE). The challenge with these differentiable frameworks is that they invariably demand multiple predictions and gradient descent steps, resulting in extensive runtime. In contrast, DAGSizer [16] directly predicts gate sizes using graph convolutional networks (GCNs), which are effective at capturing local information. However, GCNs reliance on local node aggregation limits their capacity to capture crucial path-level features such as timing and power. In contrast to these GNN-based approaches, TransSizer [17] adopts sequence modeling with a transformer, predicting gate sizes in an autoregressive manner. This approach effectively captures path-level features. However, a key vulnerability of this autoregressive approach is its susceptibility to error propagation, where early mispredicted sizes can negatively impact subsequent size predictions. In addition, representing timing paths as homogeneous graphs or sequences can neglect pin information and potentially overlook important relationships between circuit elements.

From these points of view, we propose a novel heterogeneous graph-based gate sizer, dubbed *DPH-Sizer*. Specifically, we propose directly predicting gate sizes rather than relying on RL, gradient-based optimization, or autoregressive prediction. Additionally, we utilize heterogeneous graphs, which separately

represent cells and pins, instead of homogeneous graphs. To top it off, we propose Inter-Cell and Intra-Cell Graph Attention Network (GAT) [20] blocks to explicitly capture both intra-cell and inter-cell features. Finally, we incorporate transformer blocks to capture path-level features by applying the attention mechanism globally, in contrast to the local operations typically performed in GNNs. We evaluate DPH-Sizer against previous ML-based works [16, 17] across various designs using a 22nm node. Among prior ML models, DPH-Sizer demonstrates superior performance in terms of timing, power, and area. Besides, we also verify the effectiveness of each component, including the usage of heterogeneous graphs and the combination of Intra/Inter-Cell GAT blocks and transformer blocks.

In this paper, we present three key contributions.

- We propose a novel ML-based heterogeneous graph gate sizer to produce more powerful and fast sizing outcomes.
- We prove various components, including the usage of heterogeneous graphs and the combination of Inter-Cell and Intra-Cell GAT blocks and transformer blocks, through concrete ablation studies.
- We set a new state-of-the-art performance among previous ML-based works, maintaining power consumption within 2.0% on average while achieving improvements of 54.3% in timing (WNS) and 1.3% in area metrics.

The remainder of this paper is organized as follows. Section II reviews the gate sizing works, heterogeneous graph representation, and transformer architecture. Section III presents the proposed framework in detail. Experimental results and analyses are provided in Section IV. Finally, Section V concludes the paper.

II. PRELIMINARIES

A. Gate Sizing

Gate sizing involves modifying the drive strength of logic gates in a digital circuit to optimize PPA. By upsizing or downsizing, designers can speed-up timing, reduce power usage, and save chip area. However, this comes with trade-offs: upsizing can improve timing but increase power, while downsizing saves power at the cost of timing performance.

Due to the discrete nature of choices, interdependent timing constraints, exponential solution space, and multiple design objectives, gate sizing is fundamentally a combinatorial optimization problem and is classified as NP-hard. Early approaches relied on geometric programming, greedy heuristics, and dynamic programming. Recent advances in deep learning have opened a new paradigm for gate sizing. In particular, [13–16, 18, 19] highlight the use of graph representations and graph neural networks (GNNs) for this task. The relatively long runtimes of gradient-based optimization methods [14, 15], as well as RL-Sizer [13] and TransSizer [17], have motivated the shift toward direct prediction approaches.

B. Heterogeneous graph

Graphs have been extensively utilized for representation across various domains, including social networks [21, 22], molecular and biological systems [23], and even circuits [24]. Recently, their application in circuits has highlighted the strong potential of graph representations when combined with GNNs, enabling advancements in tasks such as gate sizing [13–16, 18,

19], layout decomposition [25], and power estimation [26, 27].

Homogeneous graphs treat cells and their pins as a single node, ignoring critical information such as capacitance, slew, and slack, which can vary across pins even within the same cell. In contrast, heterogeneous graphs distinguish between pin nodes and cell nodes, allowing them to encode detailed information for each pin and cell. This enables precise modeling of timing paths, which is essential for gate sizing. This fundamental advantage leads us to choose a heterogeneous graph over a homogeneous one.

C. Transformer

The transformer [28] architecture, along with ResNet [29], is considered one of the highest-priority choices in modern deep learning architectures. While transformer originally introduced for neural machine translation [28], the transformer has been widely adopted across various domains, including computer vision [30], audio processing [31, 32], and autonomous driving [33, 34], due to its strong representation capabilities and global operation.

At the heart of the transformer architecture lies the attention mechanism, which computes attention scores between all token pairs, rather than only between adjacent elements as in CNNs and GNNs. This mechanism serves as the core operation in both the encoder and decoder blocks. It can be mathematically formulated as follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V, \quad (1)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad (2)$$

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V), \quad (3)$$

where Q , K , and V denote the query, key, and value matrices, respectively; d_k represents the dimension of these matrices, and h indicates the number of attention heads. For each head i , the input X is projected into separate sub-spaces via learned projection matrices W_i^Q , W_i^K , and W_i^V , allowing each head to capture distinct features from the input.

In this work, we use transformer encoder blocks at the end of GNN (Inter-Cell and Intra-Cell GAT) blocks to capture path-level features beyond local connection-level information.

D. DPH-Sizer VS ML baselines

Method	Data	Direct Sizing	Global	Local
[13]	Homo	X	△	O
[18]	Hetero	X	△	O
[16]	Homo	O	△	O
[17]	Sequence	X	O	△
DPH-Sizer	Hetero	O	O	O

* O: Fully supported; X: Not supported; △: Partially supported or uncertain.

TABLE I: Comparison between DPH-Sizer with previous ML baselines [13, 16–18] in terms of data type, direct sizing, global interaction and local interaction.

Machine learning (ML) has been increasingly adopted for the gate sizing problem over the past five years. Existing methods can be categorized based on several criteria: (1) the type of input representation—homogeneous graph, heterogeneous graph, or sequence; (2) whether they directly predict the gate size in

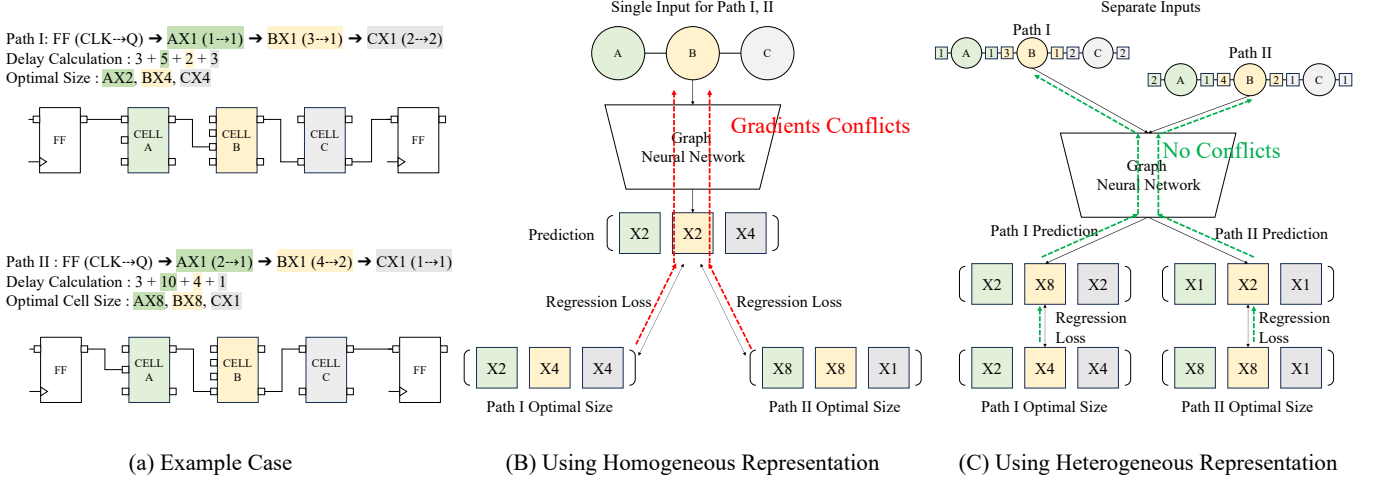


Fig. 2: Why is a heterogeneous graph representation necessary?

a given path; and (3) whether their operations rely on local or global aggregation. Here, direct sizing refers to approaches that produce gate size predictions in a single forward pass, without requiring iterative updates or repeated inference steps. These comparisons are summarized in Table I.

Data representation. It is essential to represent timing paths distinctly while preserving all relevant features that may influence sizing decisions. One crucial design choice is between graph-based and sequence-based representations. For example, cell sequences, as used in [17], enumerate only the cell tokens within a path and are inherently limited in their ability to capture the connectivity between adjacent cells. Moreover, such sequence models cannot account for the influence of cells outside the selected path, even though these cells may significantly impact the timing of closure.

Another key consideration is the graph-based representation for circuit connectivity. For example, a homogeneous graph adopted in [16] collapses each cell and its associated pins into a single node. While this simplifies the connectivity, it discards detailed connectivity information, such as which specific pin of one cell is connected to which pin of another. This abstraction can lead to multiple semantically distinct paths with same cell order being represented identically, resulting in conflicting gradient signals during training, as illustrated in Figure 2.

In contrast, a heterogeneous graph representation, which treats cells and pins as distinct entities, preserves finer-grained connectivity and pin-specific relationships. The differentiation between pins and cells is essential, as methods that fail to make this distinction encounter several issues. First, when only cells and on-path pins are represented as nodes, critical information from worst-slack pins, which are key to gate sizing decisions, may be omitted, resulting in inaccurate sizing. Second, embedding all pins as nodes requires extensive padding due to the varying number of pins per cell, which introduces computational inefficiency and can adversely affect model learning and accuracy.

Global and Local Operation. GNNs, as adopted in [13, 16, 18], typically aggregate features from adjacent nodes within a local neighborhood. While this effectively captures local connectivity between cells and pins, it falls short in modeling path-level features, such as timing and power. In contrast,

transformers perform global aggregation by computing attention scores between all elements in the path, enabling them to capture global path-level features. However, transformers inherently lack of ability to effectively model fine-grained local dependencies, being locally agnostic [38]. Driven by this complementarity, DPH-Sizer combines both GNNs and transformers to effectively capture both local and global features.

Direct Sizing. Methods [13, 16–18] require multiple inference steps, which often lead to long runtimes—particularly in the case of [13]. To address this issue, we adopt a direct sizing approach that predicts the gate sizes for a given path in a single inference step, significantly reducing overall runtime.

III. DPH-SIZER

In this section, we describe the implementation of DPH-Sizer in terms of data representation (Section III-A), model architecture (Section III-B), and the loss function (Section III-C).

A. Data Representation

1) *Heterogeneous Graph:* We begin by extracting input-to-register, register-to-register, and register-to-output paths from the entire design. A heterogeneous graph is then constructed by combining paths that share the same source and destination pins into a single graph. We can express the graph representation as follows:

$$\mathcal{G}_{s,t} = (V_{s,t}, E_{s,t}) = \left(\bigcup_{p \in \mathcal{P}_{s,t}} V_p, \bigcup_{p \in \mathcal{P}_{s,t}} E_p \right), \quad (4)$$

$$V_{s,t} = V_{s,t}^{\text{cell}} \cup V_{s,t}^{\text{pin}}, \quad E_{s,t} = E_{s,t}^{\text{gate}} \cup E_{s,t}^{\text{net}} \cup E_{s,t}^{\text{cell}}, \quad (5)$$

where $\mathcal{P}_{s,t}$ denotes the set of all paths originating at source node s and terminating at sink node t . $\mathcal{G}_{s,t}$ is a heterogeneous graph with source node s and sink node t . The node set $V_{s,t}$ is composed of two types of nodes: cell node $V_{s,t}^{\text{cell}}$ and pin nodes $V_{s,t}^{\text{pin}}$. Similarly, the edge set $E_{s,t}$ consists of three types of edges. The first edge type is the gate arc edge $E_{s,t}^{\text{gate}}$, which is the connection between a cell and its corresponding pins. The second edge type is the net arc edge $E_{s,t}^{\text{net}}$, representing connections between pins belonging to different cells. These edges correspond to physical net connections, capturing the driver-load relationships and effectively modeling signal propagation

across the circuit. Finally, the cell arc edge $E_{s,t}^{\text{cell}}$ represents internal connections within a cell.

2) *Embedding Features*: We extract physical and timing information and embed these features into each pin and cell node as described in Table II. The features embedded in the cell nodes are denoted as f_{cell} , while those in the pin nodes are denoted as f_{pin} .

The f_{cell} extracted from the circuit are as follows:

- *Coordinates*. The physical coordinates (llx , lly , urx , ury) of cells are closely related to the timing of closure.
- *Library cell properties*. The library cell properties, such as area, leakage power, and FO4 delay are the basis for gate sizing.
- *Number of pins*. The number of pins of a cell indicates logic complexity and fan-in/fan-out characteristics.
- *function index*. The function index indicates the logical functionality of the cell, which impacts gate sizing by identifying critical components such as buffer cells.

The f_{pin} extracted from the circuit are as follows:

- *Coordinates and direction*. Pin coordinates and direction clearly indicate signal paths, allowing the model to capture flow-based gate sizing characteristics effectively.
- *Rise/fall capacitance of pin*. Rise/fall capacitance directly determines delay and slew, enabling the model to effectively understand electrical load conditions critical for timing optimization.
- *Rise/fall slack of pin*. Rise/fall slack highlights timing margins, enabling the model to recognize critical paths essential for timing-driven gate sizing.
- *Rise/fall slew of pin*. Since rise/fall slew critically impacts timing and signal integrity, embedding slew information ensures sizing decisions meet slew constraints.

TABLE II: Pin and Cell Node Features.

Type	Features	Size
Cell	coordinate of bounding box	4
	library cell properties	4
	number of pins	2
	function index	1
Pin	coordinate and direction of pin	3
	capacitance characteristics	4
	timing metrics	12

B. Model Architecture

DPH-Sizer consists of Intra-Cell GAT, Inter-Cell GAT blocks, and transformer blocks. Notably, each of these GAT-based components, integrated with node aggregation, forms a distinct HeteroGAT block. DPH-Sizer leverages two HeteroGAT blocks and two transformer blocks. In our architecture, the FC layer transforms the input features into a 128-dimensional representation. The entire flow of our architecture is illustrated in Figure 3. Each GAT layer is set with a 256-dimensional hidden dimension, and the transformer layer is configured with a 1024-dimensional feed-forward network. Both the GAT and transformer layers are configured with 4 heads and a 0.3 dropout rate to mitigate overfitting, while also utilizing the GELU [35] activation function. Each component has its own roles as described below.

Intra-Cell GAT Block. In order to capture intra-cell extract features, three independent graph attention network (GAT) layers are deployed in parallel. The first layer operates on the cell-to-pin gate arc edge $E_{s,t}^{\text{gate}(c \rightarrow p)}$, aggregating library cell information and drive strength from the cell node to its associated pin nodes. The second layer operates on the pin-to-cell gate arc edge $E_{s,t}^{\text{gate}(p \rightarrow c)}$ to aggregate pin-level timing features, such as slew and arrival time, back to its parent cell node. The third layer operates on the cell arc edge $E_{s,t}^{\text{cell}}$, bidirectionally propagating spatial and timing information between the input and output pins of the same cell. The formula of the embedding update of vertex v is as follows:

$$\mathbf{h}_{(l+1),v}^{\text{intra}} = \sum_{r \in \mathcal{R}_{\text{intra}}} \text{GAT}_{(l)}^r(\{\mathbf{h}_{(l),u}^{\text{intra}} : u \in \mathcal{N}_v^r\}), \quad (6)$$

$$\mathcal{R}_{\text{intra}} = (E_{s,t}^{\text{gate}(c \rightarrow p)}, E_{s,t}^{\text{gate}(p \rightarrow c)}, E_{s,t}^{\text{cell}}), \quad (7)$$

where $\mathbf{h}_{(l),v}^{\text{intra}}$ is the Intra-Cell GAT layer- l embedding of node v , and $\mathcal{N}_v^r$ is the neighbors of v connected by edge relation r .

Inter-Cell GAT Block. This module is designed for extracting inter-cell features between connected cells. Graph attention operation is applied to the net-arc edge set $E_{s,t}^{\text{net}}$, which connects driver pins to their corresponding load pins in different cells. Attention-based aggregation along these edges bidirectionally propagates timing and electrical features, such as driver/load-pin capacitance and timing metrics, between each driver cell and its load cells. The formula of the embedding update of vertex v is as follows:

$$\mathbf{h}_{(l+1),v}^{\text{inter}} = \sum_{r \in \mathcal{R}_{\text{inter}}} \text{GAT}_{(l)}^r(\{\mathbf{h}_{(l),u}^{\text{inter}} : u \in \mathcal{N}_v^r\}), \quad (8)$$

$$\mathcal{R}_{\text{inter}} = E_{s,t}^{\text{net}}, \quad (9)$$

Node Aggregation. After passing through the two types of independent GAT blocks, the resulting embedding vectors are aggregated at the node level via summation to obtain the final node representations. The final embedding of vertex v at layer l is updated as follows:

$$\mathbf{h}_{(l),v} = \mathbf{h}_{(l),v}^{\text{intra}} + \mathbf{h}_{(l),v}^{\text{inter}} \quad (10)$$

Transformer Block. By incorporating both Inter-Cell and Intra-Cell GAT blocks, DPH-Sizer effectively captures local interactions within each cell as well as between adjacent cells. However, since the gate sizing problem typically requires consideration of whole cells of the path to optimize path-level metrics such as timing, we introduce a stack of two transformer blocks after the node aggregation stage. These transformer blocks compute attention globally, regardless of whether the cells are directly connected or not. To encode topological information, we employ learnable positional encodings, which help the model effectively capture the relative positions of cells within a given path.

C. Loss Function

How do we encourage the model to predict only valid sizes? We formulate gate sizing as a node classification task. A key challenge is that each gate has a distinct set of valid sizes, so predictions after training may fall outside the permissible range for some gates. For example, an inverter has 16 possible sizes, whereas a 2-input NAND gate has 8 in our PDK.

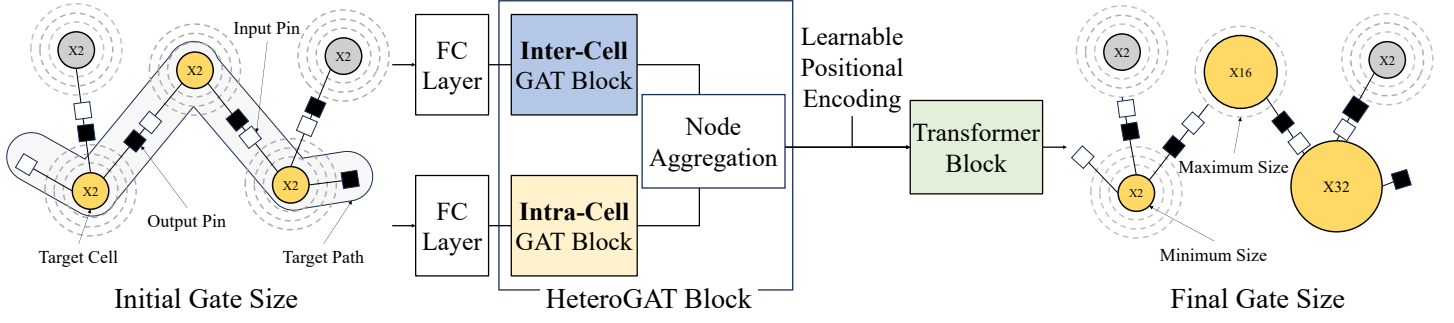


Fig. 3: Illustration of DPH-Sizer Architecture.

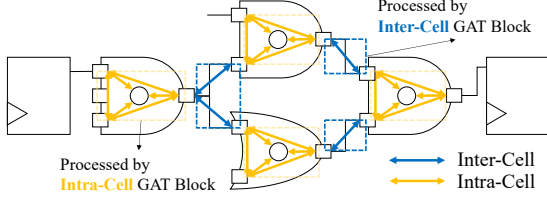


Fig. 4: Role of Intra/Inter-Cell GAT Block.

To address this, DPH-Sizer incorporates a penalty term into the original cross-entropy loss during training. This penalty encourages the neural network to predict within the valid size range rather than selecting invalid sizes. However, during inference, the predicted size may still fall outside the valid set. To handle this, we apply a clipping technique: if a predicted size exceeds the permissible maximum, it is clipped to that maximum value.

$$\mathcal{L}_{\text{valid}} = - \sum_{c=1}^L M_{i,c} \cdot y_{i,c} \cdot \log(q_{i,c}), \quad (11)$$

$$M_{i,c} = \begin{cases} 1, & c \notin C_i^{\text{valid}} \\ 0, & \text{otherwise} \end{cases}, \quad (12)$$

where L is the total number of available gate-size classes, $y_{i,c} \in \{0, 1\}$ is the one-hot label for gate i , $q_{i,c}$ is the softmax probability predicted by the model, and C_i^{valid} is the set of valid gate size for gate i .

How do we prevent large leakage? In addition to the penalty term, we explicitly multiply the leakage value of the predicted size with the cross-entropy loss. This ensures that the gradient signal is amplified when the predicted gate size results in high leakage.

$$\mathcal{L}_{\text{leakage-aware}} = - \sum_{c=1}^L \text{Leak}_{i,c} \cdot y_{i,c} \cdot \log(q_{i,c}), \quad (13)$$

where $\text{Leak}_{i,c}$ denotes the leakage power of gate i when sized to class c .

Total Loss Function. Our final loss function can be expressed as follows:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{valid}} + \alpha \mathcal{L}_{\text{leakage-aware}}, \quad (14)$$

where α is a hyperparameter that controls the strength of the leakage reduction term.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

The experiments are conducted on a server with an *Intel Xeon Gold 6226R* CPU and an *NVIDIA TITAN RTX* GPU. We utilize open-source benchmarks provided by *OpenCores* [39]. The dataset consists of five designs as described in Table III. To evaluate DPH-Sizer and baseline methods [16, 17], we use 4 designs for training and the remaining one for testing. This process is repeated 5 times, with each design serving as the test set once.

Synthesis is performed using *Synopsys Design Compiler* [40], and placement and routing stage is carried out with *Cadence Innovus* [41] using the *Intel 22nm* PDK. The PPA of sizing results is also extracted in *Cadence Innovus*. The DPH-Sizer is implemented using *PyTorch* [42] and *DGL* [43]. For optimization, we use the Adam [44] optimizer with an initial learning rate γ of 10^{-3} and decay factor $\Delta\gamma$ of 0.5. If the loss does not improve over 10 consecutive epochs, we further reduce the learning rate by a factor of 10^{-1} until it reaches a minimum of 10^{-8} . We train each model for a total of 100 epochs.

TABLE III: Design statistics for gate sizing.

Index	Design	# Cells	# Nets	# Pins
D1	<i>wb_conmax_top</i>	23745	24876	88441
D2	<i>eth_top</i>	30957	31111	111663
D3	<i>Rocket</i>	18967	65621	62273
D4	<i>des3</i>	63248	19489	200366
D5	<i>jpeg_encoder</i>	57365	62542	190496

B. PPA Analysis

1) *Comparison with ML-based Sizer:* Table IV compares PPA metrics between DAGSizer [16], TransSizer [17], and DPH-Sizer. Results demonstrate that DPH-Sizer achieves superior PPA performance. The key findings from our detailed analysis are summarized as follows:

- Compared with existing best ML-based sizer [16], our proposed DPH-Sizer achieves approximately 31.5% and 54.4% improvement in WNS and TNS, respectively. This indicates that the integration of Intra/Inter-Cell GAT and transformer block effectively enhances gate sizing performance.
- Alongside timing improvements, DPH-Sizer achieves average leakage power reductions of approximately 4.9% and 2.0% compared to TransSizer and DAGSizer, respectively, indicating that our proposed loss function effectively balances timing and leakage.

TABLE IV: Comparison of gate sizing performance Results.

Design	TNS [ns]			WNS [ps]			Leakage [μW]			Area [μm^2]		
	[17]	[16]	DPH-Sizer	[17]	[16]	DPH-Sizer	[17]	[16]	DPH-Sizer	[17]	[16]	DPH-Sizer
D1	-58.5	-39.9	-34.6	-144.5	-93.8	-69.0	228.82	208.60	207.16	11.10k	10.60k	10.54k
D2	-76.2	-102.5	-62.9	-58.6	-63.8	-54.1	563.30	555.78	554.49	28.68k	28.48k	28.46k
D3	-45.3	-35.9	-25.5	-149.1	-111.9	-86.1	266.33	259.02	<u>259.58</u>	12.13k	11.94k	11.90k
D4	-34.1	-44.1	-27.9	-53.9	-109.6	-41.5	909.71	901.42	825.76	37.38k	37.22k	35.24k
D5	-3.898	<u>-0.584</u>	-0.379	-105.6	-99.7	-22.3	1070.77	1076.38	<u>1075.37</u>	40.84k	40.90k	<u>40.84k</u>
Norm. Avg	3.24	1.46	1.00	2.19	2.19	1.00	1.05	1.02	1.00	1.03	1.01	1.00

*The best results are marked in blue, and the second-best are underlined.

2) *Comparison with OpenROAD Sizer*: Table V contains the PPA of the OpenROAD Sizer [37]. For a fair comparison, we disable options such as inserting buffers¹ in the OpenROAD Sizer, so as not to change the number of cells. Under this setting, DPH-Sizer outperforms the OpenROAD Sizer by 31.0% in WNS and 4.5% in TNS, while incurring only a 12.6% increase in leakage power and a 4.1% increase in area.

TABLE V: Gate sizing results: OpenROAD Sizer

Design	TNS [ns]	WNS [ps]	Leakage [μW]	Area [μm^2]
D1	-58.4	-134.1	200.61	10.24k
D2	-330.7	-78.3	523.44	28.03k
D3	-163.3	-162.0	221.10	11.36k
D4	-64.0	-72.8	654.03	32.55k
D5	-36.4	-203.4	874.72	39.17k
Norm. Avg	22.34	3.23	0.87	0.96

C. Model Performance

In Table VI, we first compare our implementations of TransSizer [17], DAGSizer [16], Homogeneous Graph Attention Model (HomoGAT), and Heterogeneous Graph Attention Model (HeteroGAT), and DPH-Sizer. For the fair comparison, the parameters of HomoGAT and HeteroGAT are configured to be within a 5% margin compared to those of DPH-Sizer.

We summarize the result in Table VI as follow:

- Our HomoGAT model outperforms DAGSizer, a homogeneous graph, and GCN-based approach by an average accuracy of approximately 5.8%, demonstrating that GAT is more suitable than GCN for gate sizing.
- Our model utilizing HeteroGAT achieves 12.9% higher accuracy than the HomoGAT model. This indicates that employing a heterogeneous graph that distinguishes pins and cells better captures the complex relationships between circuit components, resulting in superior performance.
- Compared to HeteroGAT, DPH-Sizer achieves 6.8% higher accuracy, demonstrating that integrating a transformer significantly enhances ability to globally capturing the path-level features.

D. Runtime Analysis

Table VII presents runtime comparisons between DPH-Sizer and ML-based sizers [16, 17]. The batch size is set to 128 for all models in this experiment. DPH-Sizer achieves an 18.8× speedup compared to TransSizer, which relies on autoregressive prediction. Although DPH-Sizer is 6.98× slower than DAGSizer, it significantly outperforms TransSizer in terms of PPA,

¹We apply the commands `repair_timing` and `repair_design`.

TABLE VI: Validation accuracy comparison of models. Best results are highlighted in bold.

Design	Validation Accuracy [%]				
	[17]	[16]	Ours ¹	Ours ²	DPH-Sizer
D1	57.8	84.7	84.1	96.5	99.6
D2	63.0	83.8	87.2	92.3	99.8
D3	67.3	67.8	81.2	90.7	96.1
D4	57.2	60.8	66.5	89.3	97.2
D5	40.8	52.9	60.0	76.1	85.0
Avg.	57.2	70.0	75.8	88.7	95.5

¹HomoGAT, ²HeteroGAT

while still maintaining a reasonable runtime. This improvement is attributed to the autoregressive nature of TransSizer, which requires multiple inference steps. In contrast, both DAGSizer and DPH-Sizer adopt a direct prediction approach, producing gate sizing results with a single inference per graph. The relatively longer runtime of DPH-Sizer compared to DAGSizer is due to the integration of Transformer encoder layers, which enhance model accuracy by explicitly aggregating path-level features using the attention mechanism.

TABLE VII: Runtime comparison of models.

Design	Runtime [ms]		
	[17]	[16]	DPH-Sizer
D1	38.5k	187.3	1.4k
D2	93.8k	759.6	4.8k
D3	28.0k	187.3	1.2k
D4	36.7k	367.6	3.0k
D5	0.3k	2.9	0.2k
Avg.	39.5k	300.8	2.1k

V. CONCLUSION

In this work, we present a novel ML-based gate sizer, dubbed DPH-Sizer. DPH-Sizer leverages the strengths of heterogeneous graph representations to prevent overlapping representations of physically distinct paths that share the same cell sequence. To fully exploit the expressive power of heterogeneous graphs, which represent pins and cells as distinct node types, we introduce Intra-Cell and Inter-Cell GAT blocks. Furthermore, to overcome the local aggregation limitations of GAT layers and capture global path-level characteristics, DPH-Sizer incorporates transformer blocks at the end of the architecture. This powerful combination yields best outcomes in terms of PPA compared to existing ML-based sizers and OpenROAD Sizer. Additionally, we enhance the quality of sizing solutions by directly modifying the loss function, resulting in improved PPA metrics across all benchmarks.

REFERENCES

- [1] G. Flach *et al.*, “JiffyTune: circuit optimization using time-domain sensitivities”, *Proc. ISVLSI*, 2013
- [2] A. Sharma *et al.*, “Fast Lagrangian relaxation-based gate sizing using multi-threading”, *Proc. ICCAD*, 2012
- [3] CP. Chen *et al.*, “Fast and exact simultaneous gate and wire sizing by Lagrangian relaxation”, in *IEEE Transactions on Computer-Aided Design*, vol. 18, no. 7, pp. 1014-1025, 1999.
- [4] G. Flach *et al.*, “Simultaneous Gate Sizing and Vth Assignment using Lagrangian Relaxation and Delay Sensitivities”, *Proc. ISVLSI*, 2013
- [5] G. Flach *et al.*, “Effective Method for Simultaneous Gate Sizing and Vth Assignment Using Lagrangian Relaxation.”, in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 4, pp. 546-557, 2014.
- [6] A. Sharma *et al.*, “Fast Lagrangian Relaxation-Based Multithreaded Gate Sizing Using Simple Timing Calibrations”, in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 7, pp. 1456-1469, 2020.
- [7] D. Mangiras *et al.*, “Task-Based Parallel Programming for Gate Sizing”, in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 4, pp. 1309-1322, 2023.
- [8] D. Chinnery *et al.*, “Integrating LR Gate Sizing in an Industrial Place-and-Route Flow”, *Proc ISPD*, 2022
- [9] S. Hu *et al.*, “Gate Sizing For Cell Library-Based Designs”, *Proc. DAC*, 2007
- [10] Y. Liu *et al.*, “A New Algorithm for Simultaneous Gate Sizing and Threshold Voltage Assignment”, *Proc. ISPD*, 2009
- [11] MM. Ozdal *et al.*, “Gate sizing and device technology selection algorithms for high-performance industrial designs”, *Proc. ICCAD*, 2011
- [12] MRCM. Berkelaar *et al.*, “Gate sizing in MOS digital circuits with linear programming”, *Proc. EDAC*, 1990
- [13] YC. Lu *et al.*, “RL-Sizer: VLSI gate sizing for timing optimization using deep reinforcement learning”, *Proc. DAC*, 2021
- [14] X. Zhou *et al.*, “AGD: A Learning-based Optimization Framework for EDA and its Application to Gate Sizing”, *Proc DAC*, 2023
- [15] Y. Ye *et al.*, “Learning-driven Physically-aware Large-scale Circuit Gate Sizing”, in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024
- [16] CK. Cheng *et al.*, “DAGSizer: A directed graph convolutional network approach to discrete gate sizing of VLSI graphs”, *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 4, pp. 52:1-31, 2023.
- [17] S. Nath *et al.*, “TransSizer: A Novel Transformer-Based Fast Gate Sizer”, *Proc. ICCAD*, 2022
- [18] X. Zhou *et al.*, “Heterogeneous Graph Neural Network-Based Imitation Learning for Gate Sizing Acceleration”, *Proc ICCAD*, 2022
- [19] J. Seo *et al.*, “Improving timing & power trade-off in post-place optimization using multi-agent reinforcement learning”, *Proc. ICCAD*, 2024
- [20] P. Velickovic *et al.*, “Graph attention networks”, *arXiv preprint arXiv:1710.10903*, 2017
- [21] W. Hamilton *et al.*, “GraphSAGE: Inductive Representation Learning on Large Graphs” *Proc. NeurIPS*, 2017
- [22] B. Perozzi *et al.*, “DeepWalk: Online Learning of Social Representations” *Proc. KDD*, 2014
- [23] K. Yang *et al.*, “Neural Message Passing for Quantum Chemistry” *Proc. ICML*, 2017
- [24] Y. Ma *et al.*, “Understanding Graphs in EDA: From Shallow to Deep Learning”, *Proc. ISPD*, 2020
- [25] H. Ren *et al.*, “ParaGraph: Layout Parasitics and Device Parameter Prediction using Graph Neural Networks”, *Proc. DAC*, 2020
- [26] YC. Lu *et al.*, “A Fast Learning-Driven Signoff Power Optimization Framework”, *Proc. ICCAD*, 2020
- [27] Y. Zhang *et al.*, “GRANNITE: Graph Neural Network Inference for Transferable Power Estimation”, *Proc. DAC*, 2020
- [28] Vaswani *et al.*, “Attention is all you need”, *Proc. NeurIPS*, 2017
- [29] K. He *et al.*, “Deep Residual Learning for Image Recognition”, *Proc. CVPR*, 2016
- [30] A. Dosovitskiy *et al.*, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, *Proc. ICLR*, 2021
- [31] Y. Gong *et al.*, “AST: Audio Spectrogram Transformer” *Proc. Interspeech*, 2021
- [32] Y. Gong *et al.*, “SSAST: Self-Supervised Audio Spectrogram Transformer” *Proc. ICASSP*, 2022
- [33] A. Mehta *et al.*, “Transfuser: Imitation with Transformer-based Sensor Fusion for Autonomous Driving” *Proc. CoRL*, 2022
- [34] Z. Liu *et al.*, “BEVFormer: Learning Bird’s-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers” *Proc. ECCV*, 2022
- [35] D. Hendrycks *et al.*, “Gaussian Error Linear Units (GELUs)” *arXiv preprint arXiv:1606.08415*, 2016
- [36] Ajayi *et al.*, “OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain”, *Proc. Government Microcircuit Applications and Critical Technology Conference*, 2019
- [37] OpenROAD Gate Resizer GitHub. <https://github.com/The-OpenROAD-Project/OpenROAD/tree/master/src/rsz>
- [38] Li *et al.*, “Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting”, *Proc. NeurIPS*, 2019
- [39] *Opencores*, <https://opencores.org>
- [40] *Synopsys Design Compiler v18.06 User Guide*
- [41] *Cadence Innovus Implementation System v21.16 User Guide*
- [42] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library”, *Proc. NeurIPS*, vol. 32, 2019, pp. 1-12
- [43] M. Wang *et al.*, “Deep Graph Library: A Graph-Centric, Highly-Performant Package for Graph Neural Networks”, *arXiv preprint arXiv:1909.01315*, 2019
- [44] DP. Kingma *et al.*, “Adam: A Method for Stochastic Optimization”, *arXiv preprint arXiv:1412.6980*, 2014